

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет по лабораторной работе №3
по дисциплине «Методы тестирования
программного обеспечения»
«Статическое тестирование»

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Курочкин М. А.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Описание тестируемой программы	4
2 Статический анализ кода	5
3 Описание приложения и среды разработки	6
3.1 Группы правил	7
3.2 Запуск CppCheck на Windows	10
4 Статический анализ кода программы	11
Заключение	14
Список использованной литературы	15

Введение

Статическое тестирование — процесс анализа программного кода без его выполнения. Целью статического тестирования является выявление недочетов на ранних стадиях разработки, что позволяет сэкономить время и ресурсы.

Cppcheck — статический анализатор кода для языков C и C++, предназначенный для поиска проблем, которые не обнаруживаются компиляторами.

Дано: код программы «Генерация мультимножеств и выполнение логических действий с ними».

Требуется: изучить метод статического анализа кода, провести статический анализ кода при помощи CppCheck версии 2.20.0

1 Описание тестируемой программы

Программа должна реализовывать программу генерации бинарного кода Грея для заполнения универсума мультимножеств на заданной пользователем разрядности. На основе универсума формируются два мультимножества, мощности которых задает пользователь. В результате на экран выводятся результаты действий над множествами: объединение, пересечение, разность, симметрическая разность, дополнение.

В случае возникновения посторонних символов программа должна корректно обрабатывать ошибку и продолжать заданные действия.

Входные данные: число, являющееся мощностью универсума; число, являющееся мощностью первого мультимножества; число, являющееся мощностью второго мультимножества.

Выходные данные: мультимножества-результаты объединения, пересечения, разности, симметрической разности и дополнения.

Ограничения:

- $a \leq n, b \leq n$, где n - мощность универсума, a - мощность первого мультимножества, b - второго.
- $0 \leq n \leq 20$, где n - мощность универсума.

Спецификация представлена на рисунке 1.

Входные данные			Выходные данные	Реакция программы
n	a	b		
str или <0 или >20	число, >0 и <=n	число, >0 и <=n	Сообщение на экране: "Invalid letter! Please enter integer n:"	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение
число, >=0 и <=20	str или <0 или >n	число, >0 и <=n	Сообщение на экране: "Invalid letter! Please enter integer a:"	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение
число, >=0 и <=21	число, >0 и <=n	str или <0 или >n	Сообщение на экране: "Invalid letter! Please enter integer b:"	Повторный запрос ввода от пользователя до тех пор, пока не будет введено корректное значение
число, >=0 и <=22	число, >0 и <=n	число, >0 и <=n	Multiset A, B is generated successfully Union of A and B: Intersection of A and B: Addition to A relative to universum: A/B: A symmetric dif B :	Вывод результатов всех операций, завершение программы с кодом 0

Рис. 1. Спецификация

2 Статический анализ кода

Статический анализ кода выполняется без запуска программы. Он анализирует исходный код, чтобы выявить потенциальные ошибки, уязвимости и нарушения стандартов кодирования. Этот вид анализа позволяет разработчикам обнаруживать проблемы на ранних стадиях разработки, что может существенно сократить затраты на исправление проблем в будущем.

Инструменты для статического анализа могут быть настроены для проверки соответствия кода определенным стандартам или стилю, что помогает избежать распространенных проблем и улучшить читаемость кода. Это особенно важно в командах, где несколько разработчиков работают над одним проектом.

Многие инструменты для статического анализа могут быть интегрированы в процесс сборки, что позволяет автоматизировать проверку кода. Это обеспечивает постоянный контроль качества и помогает разработчикам сосредоточиться на написании кода, а не на его проверке.

Виды статического тестирования:

1. Статический анализ кода

Этот вид статического тестирования включает в себя анализ исходного кода. Основная цель — выявление потенциальных проблем, неправильных практик, структурных аномалий и нарушений стандартов кодирования.

2. Обзоры кода

Этот вид статического тестирования включает в себя анализ кода членами команды разработки или экспертами. Обзоры кода позволяют выявлять проблемы и несоответствия стандартам. Они также способствуют обмену знаниями и опытом между членами команды.

3. Анализ архитектуры

При этом виде тестирования анализируется архитектура ПО, включая структуру, зависимости между компонентами и соответствие архитектурным принципам. Это позволяет выявить проблемы, связанные с проектированием системы.

3 Описание приложения и среды разработки

Cppcheck — статический анализатор кода для языка C/C++, предназначенный для поиска ошибок, которые не обнаруживаются компиляторами. Он позволяет выявлять различные проблемы, уязвимости и возможности для оптимизации, при этом не требуя сборки иди запуска программы. Проверка кода может быть запущена как через командную строку, так и с помощью графического интерфейса (GUI).

Одной из основных свойств CppCheck является анализ кода на наличие проблем и уязвимостей. CppCheck классифицирует их по степени серьезности:

- **Error**: серьезные проблемы, которые могут привести к сбоям программы.
- **Warning**: потенциальные проблемы, которые могут вызвать некорректное поведение программы.
- **Style**: проблемы, связанные с нарушением стиля кодирования, которые не влияют на выполнение программы, но могут ухудшить читаемость кода.
- **Performance**: рекомендации по улучшению производительности кода.
- **Portability**: проблемы, связанные с переносимостью кода на разные платформы.
- **Information**: информационные сообщения, не связанные с несоответствиями общим договоренностям.

Программист может проверить код на наличие всех классов проблем, используя параметр `enable_all`. Пример команды: `cppcheck -enable=all -inconclusive путь_к_исходным_файлам`

Для включения просмотра определенных видов проблем, нужно использовать конкретный параметр вместо `all`:

- `style` для проверки стиля кода.
- `performance` для проверки производительности.
- `portability` для проверки переносимости кода.

- `information` для получения информационных сообщений.
- `unusedFunction` для поиска неиспользуемых функций.
- `warning` для отображения предупреждений.

CppCheck поддерживает вывод результатов анализа в текстовом и XML форматах.

Для того, чтобы включить проверку правил, необходимо использовать комбинацию из параметров `-enable` и `-supress`, где `-enable` отвечает за включение всех правил из заданной категории, а `-supress` - за подавление заданных правил из категории.

3.1 Группы правил

Cppcheck проверяет следующие группы правил:

1. 64-bit portability - некорректные операции с указателями и целыми числами на 64-битных системах.
2. Assert - побочные эффекты в `assert`, вызывающие различное поведение в `debug` release.
3. Auto Variables - ошибки, связанные с временем жизни локальных переменных и указателей на них.
4. Boolean - некорректные операции с `bool`.
5. Bounds usage - проверка использования библиотеки Boost.
6. Bounds checking - выход за границы массива, переполнение указателей.
7. Check function usage - отсутствие `return` в `non-void` функциях, игнорирование возвращаемого значения.
8. Class - ошибки в классах: неинициализированные члены, отсутствие конструктора копирования, неверное наследование.
9. Condition - ошибки в условиях: лишние проверки, неправильные `if/else`.
10. Exception Safety - проверка безопасности исключений.

11. IO using format string - ошибки форматирования в printf/scanf, работа с файлами после закрытия.
12. Leaks (auto variables) - поиск утечек памяти в автоматических переменных.
13. Memory leaks (address not taken) - поиск утечек памяти, связанных с неиспользуемыми адресами.
14. Memory leaks (class variables) - поиск утечек памяти в переменных класса.
15. Memory leaks (function variables) - поиск утечек памяти в переменных функции.
16. Memory leaks (struct members) - поиск утечек памяти в структурах.
17. Null pointer - разыменование nullptr, арифметика с nullptr.
18. Other - прочие проверки.
19. STL usage - анализ использования стандартной библиотеки шаблонов STL.
20. Sizeof - проверка корректности использования оператора sizeof.
21. String - некорректное использование строковых функций.
22. Type - ошибки приведения типов данных.
23. Uninitialized variables - использование неинициализированных переменных.
24. Unused functions - поиск неиспользуемых функций.
25. UnusedVar - поиск неиспользуемых переменных.
26. Using postfix operators - анализ использования постфиксных операторов.
27. Varg - ошибки работы вариантивных функций.

Разбор CppCheck правил из группы «Type».

CppCheck предоставляет 6 проверок из группы «Type» для обнаружения неверного использования типов данных и преобразований между типами. Рассмотрим следующие правила:

- Rule: needInitialization

- Severity: warning
- Category: Type

Данное правило обнаруживает переменные, которые используются до инициализации. Это может привести к неопределённому поведению программы, так как переменная содержит случайное значение из памяти.

- Rule: hasCircularDependencies

- Severity: error
- Category: Type

Правило обнаруживает циклические зависимости между типами данных (классами, структурами). Это возникает, когда тип А содержит ссылку на тип В, а тип В содержит ссылку на тип А, что делает невозможным корректное определение размеров типов.

Разбор CppCheck правил из группы «Condition».

CppCheck предоставляет 14 проверок из группы «Condition» для обнаружения неверного использования логических условий и выражений. Рассмотрим следующие правила:

- Rule: duplicateCondition

- Severity: style
- Category: Condition

Данное правило обнаруживает ситуации, когда одно и то же условие проверяется несколько раз в пределах одной области видимости без изменения значений переменных, участвующих в проверке. Это указывает на избыточный код или логическую ошибку программиста.

- Rule: alwaysTrueFalse

- Severity: style
- Category: Condition

Правило обнаруживает условия, которые всегда истинны или всегда ложны из-за статически известных значений переменных или заранее отфильтрованных значений.

3.2 Запуск CppCheck на Windows

Через командную строку

1. Скачайте и распакуйте CppCheck.
2. Откройте командную строку (Win + R, введите cmd).
3. Перейдите в директорию с cppcheck.exe: `cd путь_к_директории`
4. Выполните анализ:
`cppcheck -enable=all -inconclusive путь_к_исходным_файлам`
Для включения просмотра определенных видов проблем, нужно использовать конкретный параметр вместо all:
 - -enable=style для проверки стиля кода.
 - -enable=performance для проверки производительности.
 - -enable=portability для проверки переносимости кода.
 - -enable=information для получения информационных сообщений.
 - -enable=unusedFunction для поиска неиспользуемых функций.
 - -enable=warning для отображения предупреждений.

Через графический интерфейс

1. Установите и откройте CppCheck GUI.
2. Укажите путь к проекту.
3. Настройте параметры анализа - выберите соответствующие опции в настройках интерфейса:
 - Стилль кода.
 - Производительность.
 - Переносимость кода.
 - Информационные сообщения.
 - Неиспользуемые функции.
 - Предупреждения.
4. Запустите анализ и просмотрите результаты в интерфейсе.

4 Статический анализ кода программы

В результате первого запуска статического анализатора были выявлены 2 стилистические ошибки и 1 предупреждение. На рисунке 2 представлен скриншот результата статического тестирования. Для каждой проблемы показана строка, тип проблемы, ее краткое описание и ID.

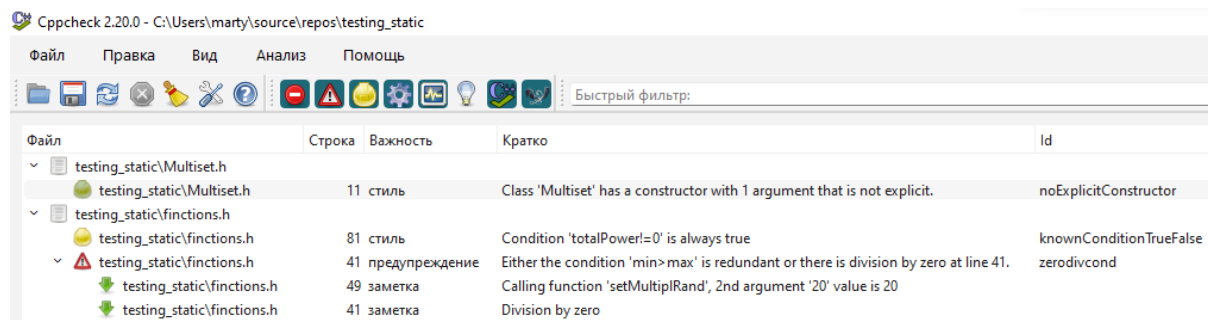


Рис. 2. Результат первого запуска программы

Описание найденных проблем:

1. Стилистические предупреждения:

- 1) Сообщение: Class 'Multiset' has a constructor with 1 argument that is not explicit.

Описание: конструктор класса Multiset принимает один аргумент и не помечен ключевым словом explicit. Это может привести к неявным преобразованиям типов, когда целое число будет автоматически преобразовываться в объект Multiset.

Решение: изменить объявление конструктора на `explicit Multiset(int size);`

```
1 //было
2 Multiset(int size) {...}
3 //стало
4 explicit Multiset(int size) {...}
```

- 2) Сообщение: Condition 'totalPower!=0' is always true. Описание: условие `totalPower!=0` в цикле `while` всегда истинно, то есть проверка в цикле `while` избыточна.

Решение: убрать проверку `while (totalPower != 0)`

```
1 //было
2 while (totalPower != 0) {
3     for (int i = 0; i < univerSize && totalPower > 0; ++i)
4     {...}
```

```

4     }
5     //стало
6     for (int i = 0; i < univerSize && totalPower > 0; ++i)
7         {...}

```

2. Предупреждение:

Сообщение: Сообщение: Either the condition 'min>max' is redundant or there is division by zero at line 41.

Описание: происходит деление по модулю на выражение (max - min + 1). При min = max анализатор предупреждает о возможном делении на ноль.

Решение: добавить проверку на равенство min и max перед выполнением операции:

```
if (min == max) return min;
```

```

1     //было
2     int setMultiplRand(int min, int max) {
3         if (min > max) { swap(min, max); }
4         return (rand() % (max - min + 1)) + min;
5     }
6     //стало
7     int setMultiplRand(int min, int max) {
8         if (min == max) { return min; }
9         if (min > max) { swap(min, max); }
10        return (rand() % (max - min + 1)) + min;
11    }

```

После исправления всех проблем CppCheck сообщает о том, что ошибок не обнаружено. На рисунках 3- 4 представлены скриншоты с результатом статического тестирования после исправления свех проблем.

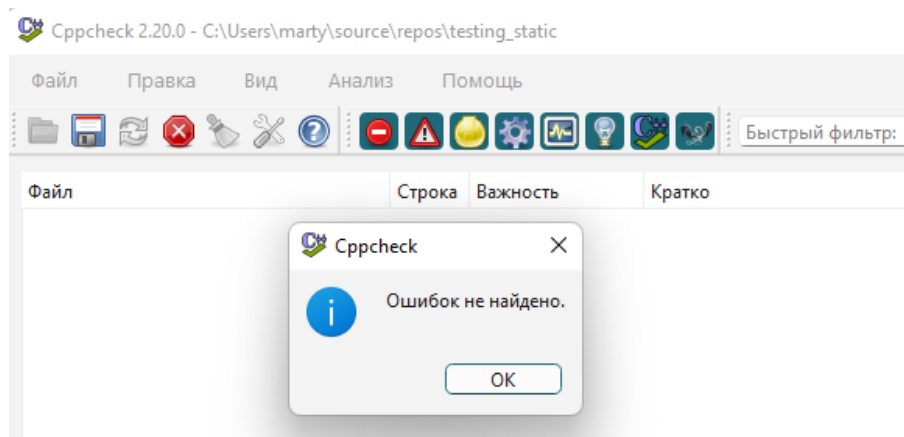


Рис. 3. Результат статического тестирования

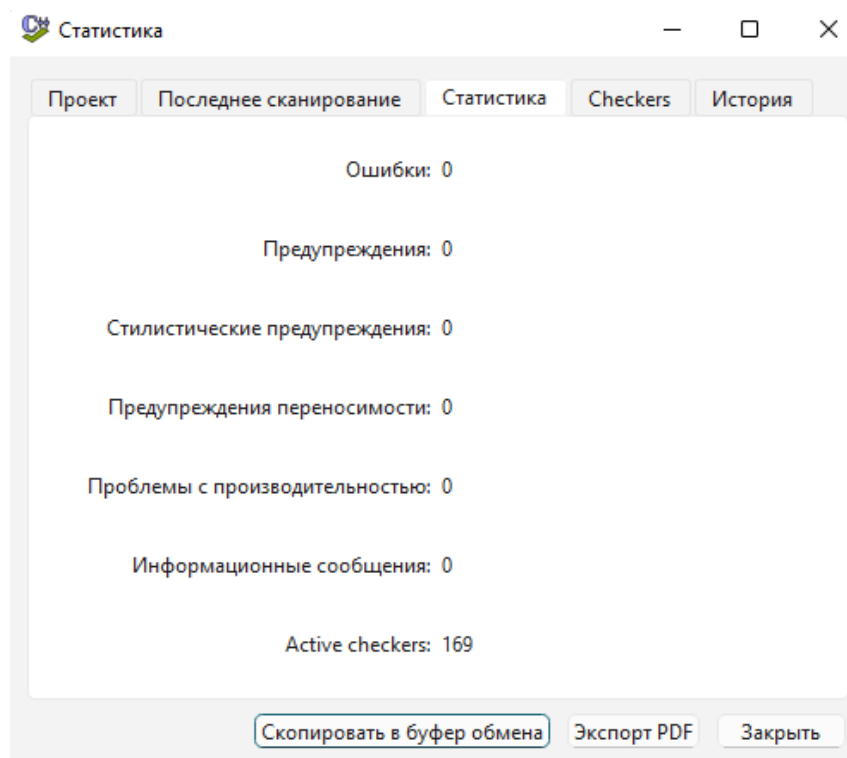


Рис. 4. Результат статического тестирования

Заключение

В ходе выполнения данной лабораторной работы был проведен статический анализ кода при помощи CppCheck версии 2.20.0, а также описано само приложение. Проведенный анализ кода программы выявил 2 стилистические ошибки и 1 предупреждение.

Ключевые недостатки были связаны с избыточным условием, возможностью деления на ноль и проблемой с неявным приведением типов из-за отсутствия в конструкторе класса ключевого слова.

Все обнаруженные проблемы были успешно устранены, что подтвердилось повторным запуском приложения CppCheck.

В результате выполненной работы можно сделать вывод, что статический анализ позволил обнаружить недочеты, пропущенные при ручной инспекции кода. В то же время ручная инспекция помогла выявить проблемы, которые статическое тестирование не обнаружило (переименование переменных и функций для увеличения осмысленности их названий). Таким образом, лучше использовать эти методы совместно для большей эффективности тестирования.

Список литературы

- [1] Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. - Изд. 3-е. - Санкт-Петербург: Диалектика, 2012 - 272 с.
- [2] CppCheck. — SourceForge [Электронный ресурс] // URL: <https://cppcheck.sourceforge.io/> (дата обращения: 14.03.2026).